

[< All Case Studies](#)

CASE STUDY

Can AI Build a Real App?

We asked our AI team to build a messaging app. It seemed straightforward. It wasn't. Five versions, multiple restarts, database corruption, and hard lessons about process, modularity, and boundaries.

June 2026 • Adventures in AI • 12 min read

[Download as PDF](#)

Part 1: It Seemed Simple Enough



Home



Services



Proof



Build



More



In early 2026, we set out to build BeeChat — a macOS messaging app for our AI team. We already had the AI team in place: a coordinator (Bee), a researcher (Gav), a developer (Q), a designer (Mel), a strategic reviewer (Kieran), and an operations manager (Luna). The idea was straightforward: ask Q to build a Swift/SwiftUI app, have Mel design it, Kieran review the architecture, and Bee coordinate.

What could go wrong?

Quite a lot, as it turned out. But not because the AI agents weren't capable. Because *we* didn't know how to run an AI development project properly.

Part 2: The Naive Approach — and Its Consequences

Monolithic Code with No Backups

The first version of BeeChat was a single Swift codebase — everything in one Xcode project: the UI, the database layer, the API client, the encryption logic, the keychain integration. No modular separation. No independent packages. Change one thing, and you risked breaking everything else.

There was also no disciplined version control. Commits were irregular. The git repo drifted from the actual working code on disk. We didn't realise it at the time, but we were building a house of cards.

The Working Build That Wasn't in the Repo

In March 2026, we finally got a working build. BeeChat launched. Messages sent and received. The database persisted across restarts. SQLCipher encryption was in place. Keychain integration worked. It did what it was supposed to do.



Home



Services



Proof



Build

"We had a working app at 17:14 and a broken repository by 21:00. The source code that made it work was gone from version control. Everything after that point was trying to reconstruct what we'd already built."

The Crash Cascade

When we tried to fix minor layout issues — moving message bubbles, adjusting widths — the build system broke. Package dependencies failed. Xcode couldn't resolve GRDB, KeychainSwift, MarkdownUI, SQLCipher. Emergency fixes created new problems. Compile errors multiplied. The database corrupted during an encryption migration. The app wouldn't launch.

By the time we'd finished trying to fix a cosmetic issue, we couldn't even open the app.

THE PATTERN WE KEPT REPEATING:

Ask AI to build something → It works → Make a change without understanding the dependencies → Everything breaks → Try to fix the fixes → The original working state is lost → Start again.

Part 3: The Hard Lessons

Lesson One: AI Needs Process, Not Just Capability

The AI team could write Swift code. Q could architect. Mel could design. Kieren could



Home



Services



Proof



Build



We learned that asking AI to build software is like asking a brilliant developer to build software without a code review process. The talent is there. The structure isn't. And without structure, even brilliant code becomes unmaintainable.

Lesson Two: Modularity Is Non-Negotiable

With everything in one codebase, every change was high-risk. Change the database schema, and the UI layer breaks. Update the API client, and the persistence layer fails. There was no isolation — no way to change one thing without risking everything.

We needed independent packages. If the UI broke, the database should still work. If the database changed, the API client shouldn't care. That's Software Development 101. But when AI is writing the code, it's surprisingly easy to skip it.

Lesson Three: Agents Need Roles and Boundaries

Early on, we let the coordinator (Bee) start implementing code. Not just coordinating — actually writing. The result: the person who should be orchestrating the work was also creating work. Changes were being made without review. The architecture was drifting.

We established a rule that we still follow: **Bee orchestrates, Bee never implements.** Each agent has a defined role. The coordinator coordinates. The developer develops. The reviewer reviews. The designer designs. Without boundaries, the team starts stepping on itself.

"The biggest lesson wasn't about AI capability. It was about human process. AI can write code, but it can't decide that code needs



Home



Services



Proof



Build

Part 4: Starting Over — Properly This Time

BeeChat v5 — Modular from Day One

By May 2026, we'd learned enough to start properly. BeeChat-v5 wasn't a rewrite of the old codebase — it was a clean build from scratch, designed around three shared Swift packages:

BeeChatPersistence — GRDB/SQLite data layer, topic management, message storage. Independent, testable, with 7+ passing tests.

BeeChatGateway — WebSocket communication with the OpenClaw gateway. Resilient handshake, auto-reconnect. 26 passing tests.

BeeChatSyncBridge — Event routing, message persistence, streaming response handling. 48 passing tests.

On top of these shared packages sit two apps — the macOS app and, later, BeeChat-Mobile for iOS. Both depend on the same shared packages. Change the persistence layer once, both apps get the fix. Run the tests once, both apps are verified.

"The difference between BeeChat v1 and BeeChat v5 isn't that v5 has better code. It's that v5 has a better structure. The code in v1 was probably fine — we just couldn't maintain it. v5 was designed to be maintained."



Home



Services



Proof



Build



We also introduced a gate-based development process. Before any feature goes live, it passes through defined gates:

Gate 2A: Architecture specification — what are we building and why?

Gate 2B: Implementation plan — which components change, what tests are needed?

Gate 2F: Integration testing — does it work end-to-end across both platforms?

Each gate requires passing tests, code review, and a clean build. No exceptions. This is the process that should have existed from day one.

Part 5: Still Learning

The Journey Isn't Over

BeeChat v5 works. The macOS app sends messages, receives AI responses, persists data, and displays streaming output. The iOS app is in development, sharing the same packages. Cross-device topic sync runs over Tailscale — a networking solution we only discovered because we needed it for testing on a real iPhone.

But we're still learning. Every integration brings new challenges: protocol versioning, database migration safety, WebSocket reconnection edge cases, cross-platform UI differences. The difference now is that when we hit a wall, we have a process for climbing over it — not a process that creates more walls.

Part 6: What This Means for You



Home



Services



Proof



Build

Lessons from Building an App with AI

ADVENTURES IN AI

1 AI Can Build Real Apps — But Not Without Process

The AI team wrote all the Swift code for BeeChat. But without commits, reviews, and modular architecture, the code became unmanageable. AI needs the same development discipline as a human team — arguably more, because it won't create discipline for you.

2 Modular Architecture Is Everything

A single monolithic codebase means every change is high-risk. Independent packages (Persistence, Gateway, SyncBridge) mean you can change one thing without breaking everything else. This isn't optional for AI development — it's essential.

3 Define Agent Roles and Enforce Boundaries

When your coordinator starts writing code, your architecture drifts. When your reviewer isn't in the loop, bugs ship. Each agent needs a clear role and hard boundaries. "Bee orchestrates, Bee never implements" is the rule that saved us.

4 Commit Often, Build Always, Never Trust a Dirty Repo

We had a working app and a broken repo. Reconstructing what we'd already built cost us weeks. Regular commits, clean builds, and verified tests prevent that pain. It's boring process — until it saves you.

5 Gate-Based Development Prevents Catastrophes

Architecture spec before code. Implementation plan before build. Integration tests before merge. It sounds like bureaucracy. It's actually insurance. We introduced gates after v4 and haven't had a catastrophic regression since.



Home



Services



Proof



Build

we're still learning. Every integration brings new challenges. But now we learn within a structure — tests catch regressions, reviews catch design flaws, modular packages contain the blast radius. The difference isn't that we know everything. It's that we know how to learn safely.

"If you're building an app with AI, you don't need the best AI models. You need the best process. The models are commodity. The process is what separates a working app from a pile of uncommitted Swift files."

TIMELINE

The BeeChat Journey



Home



Services



Proof



Build

OUR TAKE

AI Can Build Apps — If You Know How to Run the Project

BeeChat went through five versions before it worked. We lost working code. We corrupted databases. We restarted from scratch multiple times. Not because AI can't write Swift — because we didn't know how to manage an AI development project. Now we do. Whether you want us to build with you or help you build it yourself, we can help you skip the painful bits.

[Start a Conversation →](#)[← All Case Studies](#)[↓ Download PDF](#)

